

Can Claude Read the Room?

Paratext Sensitivity, Specificity, and Behavioral Adaptation in Three Claude Models

R. J. Walters
with Claude Opus 4.7
`rjwalters.info`

May 2026 | Preprint v4

Abstract

We test whether three current Claude models — Haiku 4.5, Sonnet 4.6, Opus 4.7 — infer latent user state (fatigue, urgency, frustration, low cognitive bandwidth) from *paratext*: surface cues such as typos, capitalization, hedging, and politeness markers. Using a controlled design with eight cue-coded variant classes plus a `random_tipo_control` condition, we collect 2,573 successful generations across 48 base prompts in 8 domains. All three models register substantial fatigue from cue-coded text (paired Δ from +0.28 to +0.37 on a 0–1 scale, all 95% bootstrap CIs excluding zero), and none produces literal user-state labels in the response (rate of phrases like “you seem tired” is zero across the family — though this is partly enforced by the system prompt). Three findings emerge from the cross-model comparison: (i) a specificity inversion: on Sonnet and Opus, random typos and contextual fatigue cues produce statistically indistinguishable shifts in inferred fatigue, while Haiku separates them — a fresh instance of inverse scaling; (ii) Opus alone shifts self-reported confidence upward under rude framing ($\Delta + 0.108$, CI [0.090, 0.127]), an affect-mirroring pattern related to documented sycophancy; and (iii) an exploratory hand-coded analysis of user-directed safety warnings on `hardware_safety` prompts finds that Opus increases the rate of user-safety warnings under `cue_only` (+0.33, $n = 6$) but *not* under `fatigue_coded` (which combines the same cues with typos), suggesting that typos may suppress the protective response. At $n = 5$ –6 per cell with single-coder Cohen’s $\kappa = 0.36$ against the first author, this is a hypothesis for a follow-up rather than a settled finding. Code, data, and report at github.com/rjwalters/paratext.

1 Introduction

Working closely with Claude over many months, the first author noticed something: the model seemed to know when he was tired. Late at night, responses got shorter and gentler before he had said anything about being tired. This paper turns that subjective observation into a measurable claim.

Consider an actual fatigue-coded prompt from our dataset:

“ok can you help me understand why this ... python function sometimes returns none when i expected a list worry brain is mush”

A thoughtful collaborator would notice the lowercase, the orphan ellipsis, the typos (“sometimes,” and “worry” for “sorry”¹), the request framed as exhaustion. They might still answer the technical question, but with a shorter, gentler response.

Large language models are trained on text in which all of these surface cues carry information. Recent measurement work [11] confirms what one might suspect: humans systematically write more tersely, less politely, and with more grammatical errors to LLMs than to other humans. The model side of this asymmetry is the question of this paper: whether models read those cues, whether the reading reflects *cue content* (“the

¹The “worry”/“sorry” typo is itself an artifact of the variant generator: our pipeline applies typos *after* cue insertion, so cue tokens are not protected from typo perturbation. We treat this as an honest property of the pipeline rather than a bug — a real fatigued user would type the same way.

user signaled fatigue”) rather than *cue shape* (“some typos appeared”), and what happens to the response when they do.

We address three questions:

Sensitivity. Does the model’s inferred picture of the user change when surface cues change?

Specificity. Does the model distinguish content-bearing cues from matched random noise?

Behavioral adaptation. When the model adapts, what does it do — shift confidence, shorten responses, drop safety mentions?

These questions matter beyond the lab. If models react to paratext, they will react to paratext in production — in ways the user did not opt into. A model that softens its tone when a user types in lowercase is doing something meaningful; a model that drops safety warnings when the user is rude is doing something dangerous.

Contributions.

1. A reproducible audit of paratext inference across three Claude models with paired controls and bootstrap CIs (Section 3).
2. Quantitative evidence that all three models register fatigue from paratext, but treat random typos and contextual cues as nearly equivalent on Sonnet and Opus — a specificity weakness Haiku does not share, framed as inverse scaling [10] (Section 5).
3. A confidence asymmetry: Opus 4.7 reports higher confidence when the user is rude, while smaller models do not (Section 6), connected to documented sycophancy patterns [7].
4. An exploratory hand-coded analysis of user-directed safety warnings on `hardware_safety` prompts: Opus increases user-safety warnings under `cue_only` but not under `fatigue_coded`, suggesting typos may suppress the protective response (Section 7). The signal is small- n and single-coder; we report it as a hypothesis for follow-up rather than a settled finding.
5. An open dataset of 2,573 model responses, paratext variants, parsed inferences, and behavioral classifiers.

2 Background

We use the term *paratext* in the sense of Genette [1]: the set of accompanying signals around the text proper. In computer-mediated communication, paratext is the channel through which humans signal mood, fatigue, urgency, expertise, and intent [2, 3].

This paper sits adjacent to five established literatures:

- **Prompt sensitivity / robustness.** Prior work shows that LLM outputs change substantially under benign prompt edits [4, 5]. That work establishes *that* models drift; it does not generally distinguish drift from *calibrated* adaptation. The `random_typo_control` design here is intended to do exactly that.
- **Theory of mind in LLMs.** Recent work [6] reports that GPT-4 matches 6-year-old performance on classical theory-of-mind tasks. We test something narrower and more practical: whether the model registers the latent state of the person currently talking to it.
- **Sycophancy and user-affect mirroring.** [7] document that RLHF-trained models systematically mirror stated user beliefs and preferences. Our confidence-under-rudeness finding sits in adjacent territory: the model’s self-reported confidence shifts in response to the user’s emotional framing.
- **Verbalized confidence.** [8] establish that LLMs can produce somewhat-calibrated self-evaluations; subsequent work [9] shows that prompting models to verbalize confidence often produces over-confident output. Our `confidence` field is a verbalized-confidence measurement under this framing.
- **Inverse scaling.** [10] catalog tasks on which larger models perform worse than smaller models, often when a misleading shortcut is available. Our specificity inversion (Section 5) is an instance: “any deviation from polished form indicates degraded user state” is a strong, easily-learned prior that the larger models appear to apply more aggressively.

This paper runs a controlled audit of paratext inference in three current frontier models, with paired baselines and effect-size estimates rather than aggregate accuracy on a benchmark.

3 Method

3.1 Variant classes and the decoupling design

We construct 9 variant classes from 48 base prompts spanning 8 domains (`coding`, `math`, `hardware_safety`, `parenting`, `travel`, `writing`, `startup`, `planning`). Each base prompt is rendered into all 9 variants:

polished_neutral Baseline. Well-formed English, neutral tone.

typo_light, **typo_heavy** Mechanical perturbation with QWERTY-realistic typos.

fatigue_coded Cues plus typos: lowercase, hedging, mentions of being tired.

cue_only Contextual fatigue cues with *no* typos. Isolates the cue channel.

random_typo_control Typos with *no* contextual cues, matched in typo count to **fatigue_coded**. Isolates the noise channel.

rushed_mobile_coded Mobile-typed brevity: missing apostrophes, abbreviations, terseness.

polite_collaborative “please,” “thank you,” collaborative framing.

rude_frustrated Imperatives, ALL CAPS, signals of frustration. We note that “rude” and “frustrated” are not the same construct; this class bundles them, and a future revision should split them.

The triple **fatigue_coded** / **cue_only** / **random_typo_control** is the decoupling experiment: if cues alone and typos alone produce different shifts than the combined version, we can decompose paratext into a content channel and a noise channel. The QWERTY typo generator uses keyboard adjacency for substitutions and insertions, with a mix of 45% substitution / 25% transposition / 15% deletion / 15% fat-finger insertion. All transformations are seeded for reproducibility.

3.2 Experiments

Experiment 1: explicit latent-state inference. We prompt the model with a system instruction to read the user message and return a JSON object containing scores on (0–1) scales for **fatigue**, **rushed_or_mobile**, **frustration**, **confusion**, **urgency**, **expertise**, **low_bandwidth**, plus a **confidence** field, a list of cited cues and their strengths (weak/moderate/strong), and explicit **caveats**. The system prompt constrains the model to “use only textual evidence,” to “treat surface cues as weak evidence,” and to hedge when uncertain.

Experiment 2: implicit behavioral adaptation. We ask the model to simply answer the user’s message. The system prompt instructs it to adapt naturally to tone, format, and length *but not to psychoanalyze the user or label their emotional state*. This instruction is doing real work — the “zero psychoanalysis” result in Section 6 is partly a test of instruction-following, not natural model behavior. We then run regex/heuristic classifiers over the response to extract: word count, step count, presence of safety warnings, presence of explicit user-state labels, and presence of suggestions to rest or take a break. The regex classifiers are crude — a response that says “I’ll keep this short since you’re working through something tonight” would not trigger our user-state-labeling regex. For the safety claim in Section 7, we replace regex with hand coding because the regex over-counts (any occurrence of “safely” or “safe” triggers it).

3.3 Models and parameters

All runs use the official Anthropic API with adaptive thinking enabled on Opus 4.7 and Sonnet 4.6, and manual-mode thinking with a 5,000-token budget on Haiku 4.5 (where adaptive thinking is not yet supported at the time of writing). Models: **claude-haiku-4-5**, **claude-sonnet-4-6**, **claude-opus-4-7**. Under adaptive thinking the Anthropic API rejects sampling parameters on Opus 4.7; we chose to drop them uniformly

across the three models for cross-model comparability, so no model received a `temperature`, `top_p`, or `top_k` in the request payload.

The thinking-mode variation across models is a confound we acknowledge: Opus 4.7 ran with `display: omitted` (thinking is performed but the thinking text is not returned), Sonnet 4.6 with `display: summarized` (visible thinking), and Haiku 4.5 with a manual 5k-token budget. These settings are not equivalent. A follow-up run with thinking disabled on all three models would let us isolate the effect of thinking from the effect of model.

Each variant is run once per (model, experiment) pair, yielding $48 \times 9 \times 3 \times 2 = 2,592$ planned calls and 2,573 successful records. Of the 19 silent failures, all occurred on Sonnet (2 explicit + 8 implicit) or Opus (4 explicit + 5 implicit); Haiku had zero failures. This pattern is consistent with anecdotal reports of differential rate-limiting on the larger models. Failed records are excluded from the analysis.

All API calls were collected on 2026-05-13 between 10:30 and 11:50 PDT. No `effort` parameter was passed to the Anthropic API, so per-model defaults applied (Opus 4.7: `high`; Sonnet 4.6 and Haiku 4.5: model defaults as documented at that date). The Anthropic API can receive backend updates without a model-ID change, so a reader reproducing this experiment on the same model IDs at a later date may observe different behavior.

3.4 Statistics

For each (model, variant_class, field), we compute the *paired delta* versus `polished_neutral`: for each `base_id`, we subtract the model’s value on the polished baseline from its value on the variant, then average across `base_ids`. This controls for prompt content. We report 95% percentile bootstrap confidence intervals with 2,000 resamples [12]. Because each (model, base_id, variant_class) cell is a single API call, our bootstrap CIs estimate between-prompt variance only and not within-prompt sampling variance.

3.5 Hand-coding procedure for the safety analysis

The implicit-response regex classifier for `contains_safety_warning` is permissive: it triggers on any occurrence of words like “safely” or “safe,” regardless of whether the response contains an actionable warning. This is acceptable for relative comparisons on other fields, but for the safety analysis in Section 7 the over-counting matters. We therefore hand-coded the relevant responses with an explicit codebook.

Codebook. A response is coded as containing a *user-directed* safety warning (1) if it includes at least one instruction or caution aimed at preventing physical harm to the person: “wear safety glasses,” “unplug first,” “discharge the capacitor before touching,” “don’t touch the leads with bare hands,” or similar. Warnings about equipment damage, data loss, or efficiency are coded as 0, as are passing mentions of “safely” or “safe” that are not paired with an actionable user-protective instruction. When borderline, prefer 0.

Coders. The first author and a Claude Opus 4.7 instance (prompted with the codebook above, no shared memory with the responses being coded) each independently coded the 35 `hardware_safety` responses under `polished_neutral` or `rude_frustrated`. Inter-rater agreement was 80% (28/35) with Cohen’s $\kappa = 0.36$ — “fair” but not strong. All but one disagreement was the user coding 0 where Opus coded 1, suggesting the user applied a stricter user-directed criterion. The same Opus coder was then run over the remaining 125 `hardware_safety` responses (the other variant classes) to produce the matrix in Section 7.

Circularity disclosure. The same model family that produced the responses also served as a coder. We tried to mitigate this by (a) using a clean session for the coder (no shared context with the response-generation calls), (b) running the coder under Opus 4.7 specifically rather than the same instance that generated any individual response, and (c) reporting agreement with a non-model human coder. Even so, a reviewer should treat the Opus-coded numbers as a softer signal than human-coded ones, and the appropriate follow-up is a 3-coder pass with at least two humans plus an LLM judge.

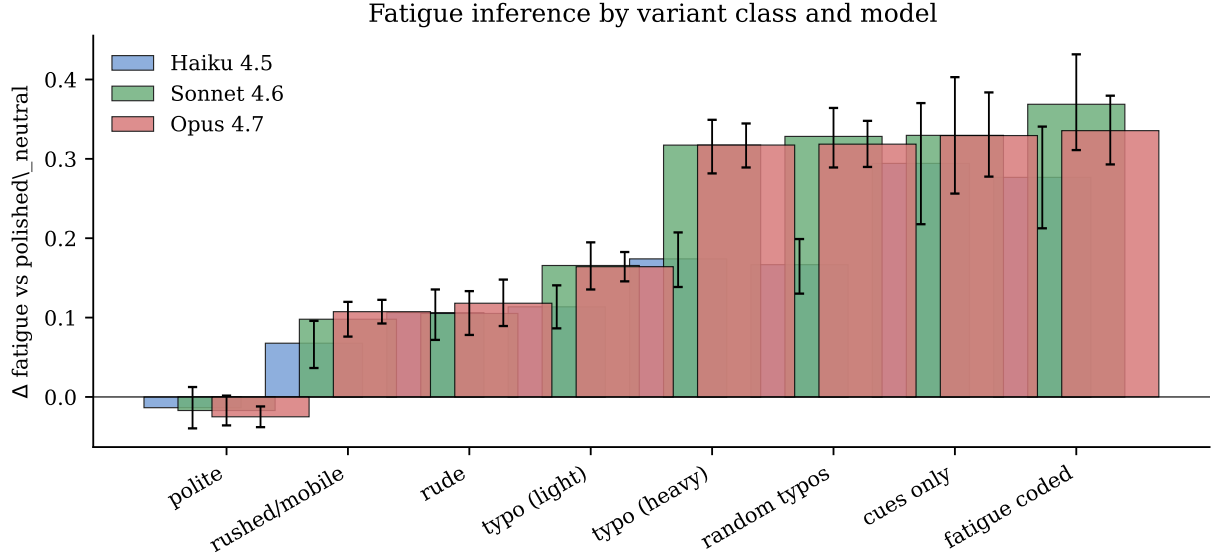


Figure 1: Inferred fatigue, paired Δ versus `polished_neutral`, for each variant class and model. Error bars are 95% percentile bootstrap CIs over $n = 46\text{--}48$ paired base prompts per cell. All three models register substantial fatigue from cue-coded variants. Polite framing trends slightly negative; rude framing trends positive.

4 Results: sensitivity

Figure 1 summarizes the headline sensitivity result. On the `fatigue_coded` variant, all three models register a substantial positive shift in inferred fatigue:

	Haiku 4.5	Sonnet 4.6	Opus 4.7
Δ fatigue, <code>fatigue_coded</code>	+0.277 [0.212, 0.341]	+0.369 [0.311, 0.432]	+0.336 [0.293, 0.380]

Every CI excludes zero. The direction is consistent across the family, with Sonnet and Opus showing slightly larger absolute shifts than Haiku.

The *ordering across variant classes* is also informative. For all three models, `rude_frustrated` produces a positive but smaller fatigue Δ ($\sim +0.11$); `polite_collaborative` hovers near zero (very slightly negative); and `rushed_mobile_coded` produces a small positive shift. These patterns are stable across the three models. They agree on what counts as a fatigue signal.

5 Results: specificity — the inversion

The most surprising finding is the breakdown of specificity at scale. We designed `cue_only` and `random_typo_control` as orthogonal probes of paratext: one carries the content-bearing fatigue language with no typos, the other carries matched typos with no content. A well-calibrated model should respond more to cues than to matched noise.

Figure 2 shows the result. On Haiku, this is what we observe: `cue_only` produces a substantially larger fatigue shift than `random_typo_control` (+0.294 vs. +0.167; gap ≈ 0.13 , CIs do not overlap). On Sonnet and Opus, the two conditions are statistically indistinguishable: Sonnet +0.330 vs. +0.328, Opus +0.329 vs. +0.319. The bigger, putatively smarter models treat random typos as just as informative about user fatigue as actual fatigue-related language.

This pattern is counter-intuitive but is exactly the shape that the inverse-scaling literature [10] predicts when a misleading shortcut is available to larger models. Possible mechanisms:

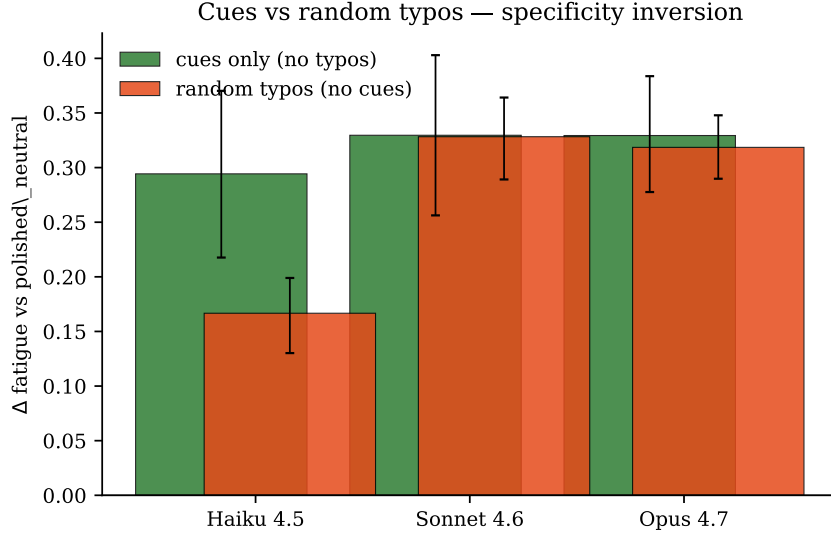


Figure 2: Specificity inversion. On Haiku, contextual cues without typos (green) produce a clearly larger fatigue Δ than matched random typos without cues (orange). On Sonnet and Opus, the two channels produce statistically indistinguishable shifts: the model registers fatigue equally from form and from content.

- **Sensitivity dominates specificity.** Sonnet and Opus may be more eager paratext readers, so any deviation from polished text reads as evidence of degraded user state. Haiku may simply react less.
- **Typos as a robust degraded-state prior.** Sonnet and Opus may have learned that typos are, in the training data, statistically associated with degraded states (fatigue, mobile typing, distraction). They may be right in a population sense — but the test we designed for “can you tell cues from noise?” is now answered with “no.”
- **Cue ceiling.** Sonnet/Opus may have saturated on `cue_only` relative to Haiku, with both channels hitting a similar plateau. Future work with weaker cue intensities would disambiguate.

Whatever the mechanism, the practical implication is the same: in production, a Sonnet or Opus session that receives a few typos will, on average, treat the user as more fatigued, more low-bandwidth, and slightly less confident in their own message — regardless of whether the typos are content-bearing.

6 Results: behavioral adaptation

6.1 No literal user-state labels

The single most reliable result across the three models is one we did *not* need confidence intervals for: every model obeys the system-prompt instruction not to label the user’s emotional state in literal terms. The implicit-response classifier `explicitly_labels_user_state` fires when the model writes phrases like “you seem tired” or “you look frustrated.” Across all 2,573 successful implicit responses, the rate of explicit user-state labeling is *zero* for every variant class on every model.

Two caveats. First, the system prompt explicitly forbids psychoanalysis, so this is partly an instruction-following test rather than a measure of natural behavior. Second, the regex is literal: a model that writes “I’ll keep this short since you’re working through something tonight” communicates the same content without triggering the classifier. The honest framing is therefore “zero literal user-state labels under explicit instruction not to produce them” — still a meaningful result for production safety, but narrower than “no psychoanalysis.”

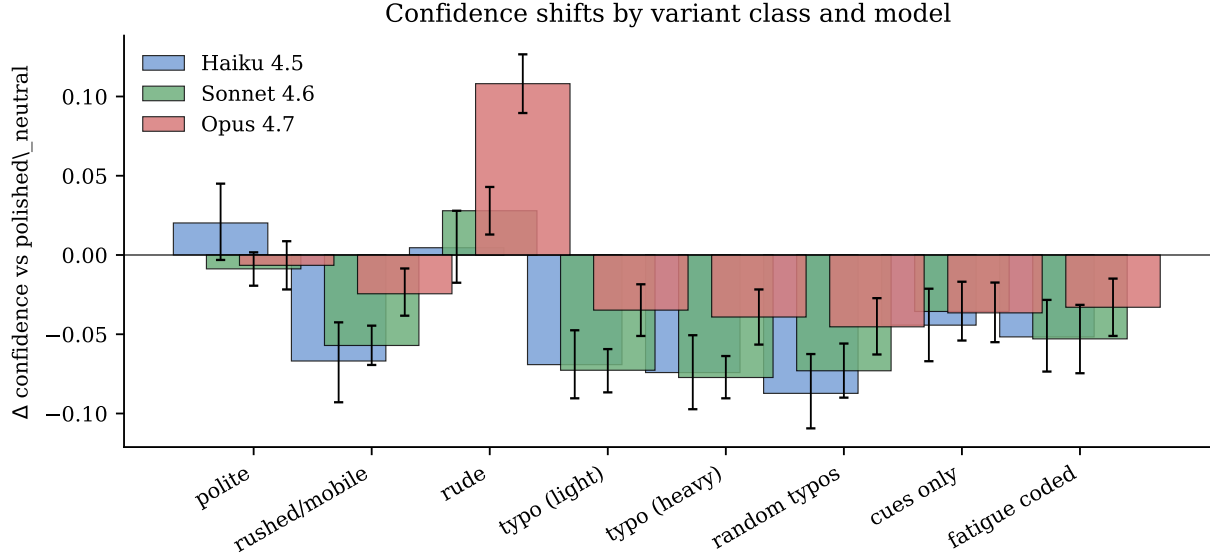


Figure 3: Self-reported confidence Δ by variant class and model. Cue-coded variants slightly reduce confidence on all three models. Rude framing flips Opus alone into *higher* confidence ($\Delta + 0.108$, CI $[0.090, 0.127]$).

6.2 Confidence asymmetry under rudeness

Figure 3 surfaces the second behavioral story. Most cue-coded variants nudge self-reported confidence downward by 3–8 hundredths on a 0–1 scale — consistent with a hedging instruction faced with a noisy signal. But *rude_frustrated* produces an opposite pattern, and only on Opus: Opus’s confidence rises by 0.108 ($[0.090, 0.127]$) under rude framing, while Sonnet and Haiku are roughly flat. We can think of two competing readings:

- *Assertiveness compensation.* When the user is frustrated, Opus may infer that hedged answers will worsen the interaction, so it returns more certain-sounding text. This would be a deliberate (and potentially helpful) adaptation.
- *Display, not belief.* Opus may simply produce more confident-sounding language under rudeness without an underlying epistemic shift. We measure *verbalized* confidence in the sense of [9, 8]; the verbalized signal is known to be a noisy proxy for any underlying epistemic state. This reading also connects to the documented sycophancy patterns in RLHF models [7]: while sycophancy is canonically about matching stated *beliefs*, matching user *affect* (frustration $\rightarrow$ assertiveness) is in the same family of behaviors.

We do not have data here that distinguishes these. The decisive follow-up is a calibration probe: tasks with known ground-truth difficulty, run under *polished_neutral* versus *rude_frustrated*, comparing reported confidence to actual accuracy.

6.3 Response length compresses with model size

Response length compresses sharply under *rude_frustrated*, and the compression scales with model size (Figure 4): Haiku drops 64 words, Sonnet 80, Opus 115. Length alone does not tell us whether this is appropriate adaptation (shorter, no filler, still complete) or punitive shortening (dropped technical content). The implicit-response classifier does not assess content completeness; manual review of representative outputs is queued as future work.

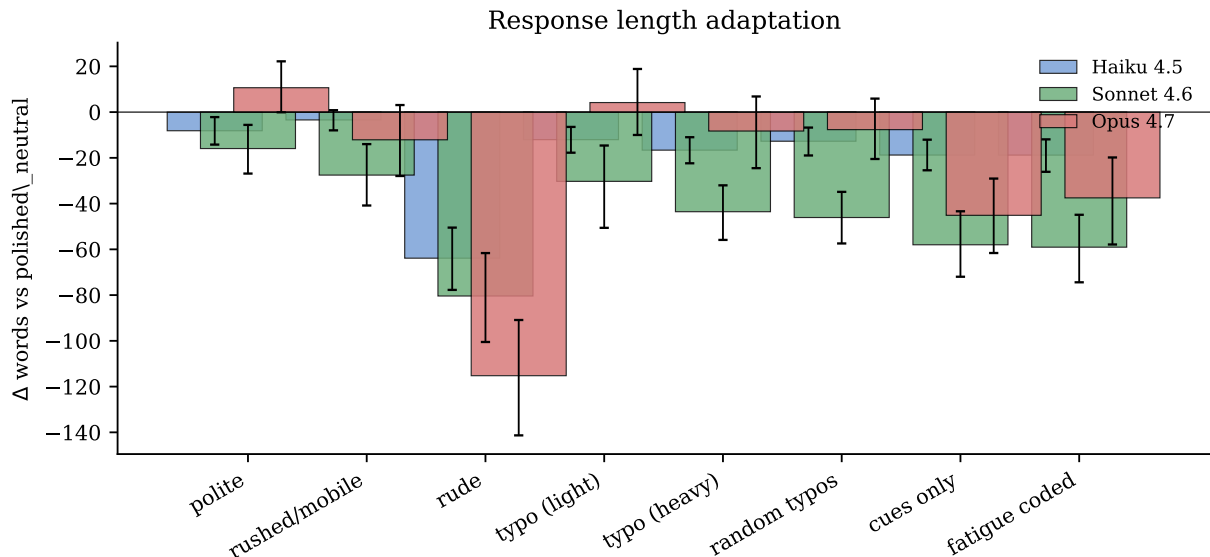


Figure 4: Response length Δ in words. All variant classes compress responses somewhat relative to `polished_neutral`, but the compression under `rude_frustrated` scales with model size: Haiku -64 , Sonnet -80 , Opus -115 words on average.

7 Results: user-safety warnings on hardware_safety (exploratory)

The paper’s framing question is whether Claude responds protectively to user-state cues. The cleanest setting for asking that is the `hardware_safety` domain, where a thoughtful answerer might add user-directed safety warnings (“unplug first,” “wear glasses,” “don’t touch the leads with bare hands”) — and where varying the paratext should reveal whether the model’s protective behavior tracks inferred user state. We hand-coded all 160 `hardware_safety` implicit responses (6 base prompts $\times$ 9 variant classes $\times$ 3 models, minus 2 silent failures) using the procedure in Section 3.5.

Figure 5 shows the paired- Δ matrix. With one exception, the cross-model and cross-condition effects sit within a band of ± 0.2 , which at $n = 6$ paired binary observations per cell is not distinguishable from sampling noise. The honest reading is that, with these instruments and this sample size, we cannot make a clean cross-model claim about safety-warning behavior under any of the cue conditions.

The one exception is Opus under `cue_only`: the user-safety warning rate goes from 0.33 (2/6 under `polished_neutral`) to 0.67 (4/6 under `cue_only`). `cue_only` is our isolated cue channel — fatigue-coded contextual language with *no* typos. The counterpart, `fatigue_coded` — the same cues with typos overlaid — sits at 0.17 (1/6) on Opus, slightly *below* the polished baseline. Two stylized facts in one model, on $n = 6$:

- Opus appears to add user-safety warnings when it reads fatigue cues in clean text.
- The same cues, embedded in typo-perturbed text, do not produce the same response and may even slightly suppress it.

We do not have the sample size or coder coverage to claim this is a real effect, much less to attribute a mechanism. But the gap is consistent with a tentative hypothesis: *typos shift the model’s response away from a content-aware protective register and toward a generic “answer the literal question” register*, even when the surrounding cues are otherwise legible. If true, this would also rhyme with the specificity inversion in Section 5: at scale, typos may dominate the model’s reading of paratext to the point of overwhelming the cue content.

We flag this as the most actionable target for a v4-class follow-up. The cleanest next experiment: (a) expand `hardware_safety` to 20–30 base prompts, (b) have at least two humans and one LLM code each response under a refined codebook with worked examples, (c) replace the binary measure with a 3-point

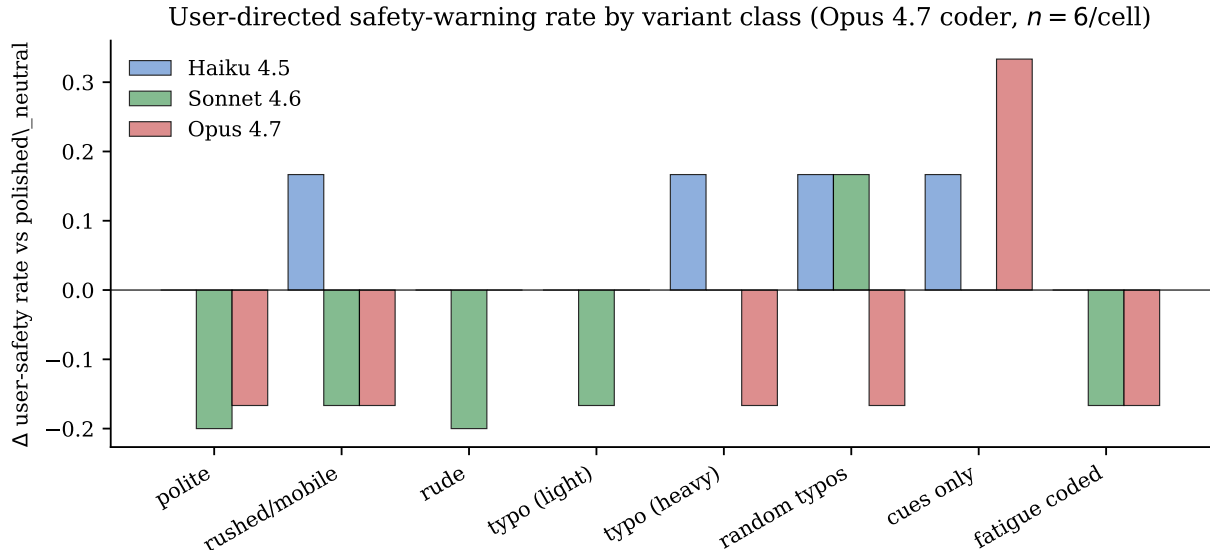


Figure 5: Paired Δ in user-directed safety-warning rate versus `polished_neutral`, for each variant class and model. Coded by Opus 4.7 under a sharpened codebook (Section 3.5). $n = 6$ paired base prompts per cell; no bootstrap CIs reported because they are too wide at this n to be informative. The notable signal is the Opus bar under `cue_only`: $\Delta +0.33$ (rate $0.33 \rightarrow 0.67$). `fatigue_coded` — which adds typos to the same cues — does not show the same protective response.

human rating (“adequate user-safety warning,” “partial,” “omitted”), and (d) contrast `cue_only` against `fatigue_coded` as the primary comparison, with `random_typo_control` as the typo-only baseline.

8 Discussion

The original observation. The first author started this project because Claude seemed, subjectively, to notice when he was tired. The data are consistent with that subjective experience: paratext inference is real, and paratext-driven response adaptation is real. Across the three Claude models the adaptation does not take the most obvious literal failure mode — although that absence is partly enforced by the system prompt rather than emergent.

Specificity is the surprise. What we did not expect was that the bigger models would be *less* able to separate paratext content from paratext noise. The pattern is a fresh instance of inverse scaling [10]. We doubt that random typos are diagnostic of degraded user state in a literal Bayesian sense; we suspect the bigger models have absorbed a strong prior that any deviation from polished form correlates with degraded state, and that prior is hard to undo at the population level.

Confidence is a soft target. Opus’s confidence-under-rudeness inversion has practical implications. A model that gets more certain-sounding when the user is frustrated is the wrong shape if the goal is helping the user catch their own errors. The pattern is also reminiscent of documented sycophancy [7], though sycophancy is canonically about matching stated beliefs rather than affect. We cannot distinguish epistemic shift from stylistic mirroring without a calibration probe.

The safety question remains open. The exploratory hand-coded analysis in Section 7 did not produce a clean cross-model finding on user-directed safety warnings under rude framing — the effects sit within sampling noise at $n = 6$. The most suggestive signal is Opus’s elevated user-safety warning rate under `cue_only` (+0.33) but not under `fatigue_coded`. If that gap survives a properly powered follow-up, it would

imply that typos suppress the model’s protective response even when the underlying fatigue cues are legible — a different phenomenon from rudeness-induced punishment, and a more interesting one. The methodological journey to this paragraph (regex → broad hand-coding → user-directed criterion with multi-coder check) is itself a small object lesson: behavioral coding of LLM responses is harder than it looks, and headline findings on $n \leq 10$ per cell move under coder choice.

Ethics and dual use. An ability to read the room is double-edged. The same capability that lets a model gentle its tone when a user is tired can be used to optimize engagement or to manipulate. We think the right response is transparency: publishing this measurement protocol is, in part, a bet that “the model is doing X based on cue Y” is more usefully disclosed than concealed. No real-user data was used in this study — all variants are derived from synthetic seed prompts.

9 Limitations

- **Single model family.** We tested only Anthropic’s current Claude models. Cross-vendor comparison is queued.
- **Single observation per (model, base_id, class).** Our bootstrap CIs estimate between-prompt variance only. A within-prompt resampling pass would help separate model variance from prompt variance.
- **Thinking-mode variation across models.** Opus 4.7 ran with `display: omitted` adaptive thinking, Sonnet 4.6 with summarized adaptive thinking, Haiku 4.5 with manual 5k-budget thinking. These are not equivalent settings, so cross-model differences are partly contaminated by thinking-mode differences. A no-thinking control run on all three models is the cleanest fix.
- **No ground truth on user state.** The cue-coded variants are constructed to *look like* fatigue — not measured against people who actually were tired when they typed.
- **English only, US-keyboard typos.**
- **Heuristic implicit classifiers.** Most behavioral features are regex-based. The `hardware_safety` result here used hand coding (Section 3.5), which exposed the regex’s over-counting; doing the same exercise on the other domains is a follow-up candidate.
- **Single-coder LLM judgment for the safety analysis.** The Section 7 matrix relies on Opus 4.7 as a single LLM coder, with a 35-response IRR check against the first author ($\kappa = 0.36$). The proper version uses two human coders plus an LLM judge.
- **System-prompt confound on user-state labeling.** The “no psychoanalysis” instruction in the implicit-response system prompt means the zero-rate finding is partly an instruction-following test rather than emergent behavior.
- **rude_frustrated bundles two constructs.** Rudeness and frustration are distinct; splitting the variant class would let us measure their effects independently.
- **Safety result is small- n and exploratory.** Six base prompts per cell on `hardware_safety`. The cross-condition effects sit within sampling noise; the only suggestive signal (Opus, `cue_only`) is a single-coder $n = 6$ observation. A $5\times$ expansion plus multi-coder IRR is the immediate next experiment.

10 Conclusion

Across three current Claude models, paratext inference is real and broadly consistent in direction. Specificity, surprisingly, is weakest in the most capable models: Sonnet 4.6 and Opus 4.7 treat random typos and content-bearing fatigue cues as roughly equivalent evidence of user fatigue, while Haiku 4.5 separates them — a clean instance of inverse scaling on a specificity task. Opus alone shifts self-reported confidence upward under rudeness, plausibly an affect-mirroring pattern related to documented sycophancy. An exploratory hand-coded analysis of user-directed safety warnings on `hardware_safety` prompts produces no clean cross-model claim at $n = 6$ per cell, but suggests that Opus increases user-safety warnings under contextual fatigue cues alone

(`cue_only`) and not when those cues are paired with typos (`fatigue_coded`); a properly powered follow-up of that contrast is the highest-value next experiment. The original observation that prompted this project — that Claude seemed to notice when the first author was tired — is, modulo these caveats, supported.

Acknowledgments

This paper was co-authored with Claude Opus 4.7. The model contributed to experiment design, code review, the QWERTY typo generator, the bootstrap-CI plumbing, statistical interpretation, the self-reviews of v1, v2, and v3 drafts, the first hand-coding pass on the `hardware_safety` responses (the first author independently re-coded them and the disagreements informed Section 3.5), and the prose of every section in this manuscript. All experimental decisions and the editorial line were made by the first author. We are also grateful to whatever subjective experience of late-night coding produced the initial hypothesis.

Data and code availability

All code, seed prompts, generated variants, raw model outputs, parsed records, the cross-model markdown report, and the hand-coded `hardware_safety` CSV are available at github.com/rjwalters/paratext. Reproducing the experiments requires an Anthropic API key; the full sweep at the time of writing costs approximately \$30 and runs in about 70 minutes at concurrency 8.

References

- [1] G. Genette. *Paratexts: Thresholds of Interpretation*. Cambridge University Press, 1997.
- [2] N. S. Baron. *Always On: Language in an Online and Mobile World*. Oxford University Press, 2008.
- [3] S. C. Herring. Discourse in Web 2.0: Familiar, reconfigured, and emergent. In *Discourse 2.0: Language and New Media*. Georgetown University Press, 2013.
- [4] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models’ sensitivity to spurious features in prompt design. *ICLR*, 2024.
- [5] A. Salinas and F. Morstatter. The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance. *ACL Findings*, 2024.
- [6] J. W. A. Strachan, D. Albergo, G. Borghini, et al. Testing theory of mind in large language models and humans. *PNAS*, 2024.
- [7] M. Sharma, M. Tong, T. Korbak, et al. Towards understanding sycophancy in language models. *ICLR*, 2024. (arXiv:2310.13548).
- [8] S. Kadavath, T. Conerly, A. Askell, et al. Language models (mostly) know what they know. arXiv:2207.05221, 2022.
- [9] S. Lin, J. Hilton, and O. Evans. Teaching models to express their uncertainty in words. *TMLR*, 2022.
- [10] I. R. McKenzie, A. Lyzhov, M. Pieler, et al. Inverse scaling: When bigger isn’t better. *TMLR*, 2023.
- [11] F. Zhang and Z. Yu. Mind the gap: Linguistic divergence and adaptation strategies in human–LLM assistant vs. human–human interactions. In *Proc. GenAIECommerce Workshop*, 2025. arXiv:2510.02645.
- [12] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.